

main.c



Share

Run

```
1 //7.binary tree
2 #include <stdio.h>
3 #include <stdlib.h>
4 // Structure for tree node
5 struct Node
6 {
7     int data;
8     struct Node *left;
9     struct Node *right;
10 };
11 // Create new node
12 struct Node* createNode(int value)
13 {
14     struct Node *newNode;
15     newNode = (struct Node*)malloc(sizeof(struct Node));
16     newNode->data = value;
17     newNode->left = NULL;
18     newNode->right = NULL;
19     return newNode;
20 }
21 // Preorder Traversal
22 void preorder(struct Node *root)
23 {
24     if(root != NULL)
25     {
26         printf("%d ", root->data);
```

```
27     preorder(root->left);
28     preorder(root->right);
29 }
30 }
31
32 // Inorder Traversal
33 void inorder(struct Node *root)
34 {
35     if(root != NULL)
36     {
37         inorder(root->left);
38         printf("%d ", root->data);
39         inorder(root->right);
40     }
41 }
42 // Postorder Traversal
43 void postorder(struct Node *root)
44 {
45     if(root != NULL)
46     {
47         postorder(root->left);
48         postorder(root->right);
49         printf("%d ", root->data);
50     }
51 }
52 int main()
```

```
53 {  
54     // Creating Binary Tree  
55     struct Node *root = createNode(1);  
56     root->left = createNode(2);  
57     root->right = createNode(3);  
58     root->left->left = createNode(4);  
59     root->left->right = createNode(5);  
60     // Traversals  
61     printf("Preorder Traversal:\n");  
62     preorder(root);  
63     printf("\n\nInorder Traversal:\n");  
64     inorder(root);  
65     printf("\n\nPostorder Traversal:\n");  
66     postorder(root);  
67     return 0;  
68 }  
69
```

Output

Preorder Traversal:

1 2 4 5 3

Inorder Traversal:

4 2 5 1 3

Postorder Traversal:

4 5 2 3 1

=== Code Execution Successful ===

```
1 //Implement a priority queue using arrays or linked lists.
2 #include <stdio.h>
3 #define MAX 5
4 int pq[MAX];
5 int size = 0;
6 // Insert element in sorted order
7 void enqueue(int value)
8 {
9     int i;
10    if(size == MAX)
11    {
12        printf("Priority Queue Overflow\n");
13        return;
14    }
15    // Insert based on priority (smaller number = higher
16        priority)
17    for(i = size - 1; i >= 0 && pq[i] > value; i--)
18    {
19        pq[i + 1] = pq[i];
```

```
19     }
20     pq[i + 1] = value;
21     size++;
22     printf("%d inserted\n", value);
23 }
24 // Delete highest priority element
25 void dequeue()
26 {
27     int i;
28     if(size == 0)
29     {
30         printf("Priority Queue Underflow\n");
31         return;
32     }
33     printf("%d deleted\n", pq[0]);
34
35     for(i = 0; i < size - 1; i++)
36     {
37         pq[i] = pq[i + 1];
```

```
38     }
39     size--;
40 }
41 // Display elements
42 void display()
43 {
44     int i;
45     if(size == 0)
46     {
47         printf("Priority Queue is Empty\n");
48     }
49     else
50     {
51         printf("Priority Queue Elements:\n");
52         for(i = 0; i < size; i++)
53         {
54             printf("%d ", pq[i]);
55         }
56     }
```

```
57     printf("\n");
58 }
59 int main()
60 {
61     enqueue(30);
62     enqueue(10);
63     enqueue(50);
64     enqueue(20);
65     display();
66     dequeue();
67     display();
68     return 0;
69 }
```

Output

30 inserted

10 inserted

50 inserted

20 inserted

Priority Queue Elements:

10 20 30 50

10 deleted

Priority Queue Elements:

20 30 50

=== Code Execution Successful ===

```
1 // Implement binary tree creation and traversal (preorder,
   inorder, postorder).
2 #include <stdio.h>
3 #include <stdlib.h>
4 // Structure for tree node
5 struct Node
6 {
7     int data;
8     struct Node *left;
9     struct Node *right;
10 };
11 // Create new node
12 struct Node* createNode(int value)
13 {
14     struct Node *newNode;
15     newNode = (struct Node*)malloc(sizeof(struct Node));
16     newNode->data = value;
17     newNode->left = NULL;
18     newNode->right = NULL;
```

```
19     return newNode;
20 }
21 // Preorder Traversal
22 void preorder(struct Node *root)
23 {
24     if(root != NULL)
25     {
26         printf("%d ", root->data);
27         preorder(root->left);
28         preorder(root->right);
29     }
30 }
31 // Inorder Traversal
32 void inorder(struct Node *root)
33 {
34     if(root != NULL)
35     {
36         inorder(root->left);
37         printf("%d ", root->data);
```

```
38         inorder(root->right);
39     }
40 }
41 // Postorder Traversal
42 void postorder(struct Node *root)
43 {
44     if(root != NULL)
45     {
46         postorder(root->left);
47         postorder(root->right);
48         printf("%d ", root->data);
49     }
50 }
51 int main()
52 {
53     // Creating Binary Tree
54     struct Node *root = createNode(1);
55     root->left = createNode(2);
56     root->right = createNode(3);
```

```
57     root->left->left = createNode(4);
58     root->left->right = createNode(5);
59     // Traversals
60     printf("Preorder Traversal:\n");
61     preorder(root);
62     printf("\n\nInorder Traversal:\n");
63     inorder(root);
64     printf("\n\nPostorder Traversal:\n");
65     postorder(root);
66     return 0;
67 }
```

Output

Preorder Traversal:

1 2 4 5 3

Inorder Traversal:

4 2 5 1 3

Postorder Traversal:

4 5 2 3 1

=== Code Execution Successful ===